

Detecting and Correcting Sensor Noise in SNOTEL Snow Depth Records: A Comparison of Statistical and Machine Learning Approaches

Anonymous

Submitted for the Degree of Master of Science in
MSc Applied Data Science



Department of Computer Science
Royal Holloway University of London
Egham, Surrey TW20 0EX, UK

September 1, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 10756

Student Name: Anonymous

Date of Submission: 1st September 2026

Abstract

Automated environmental sensor networks such as the USDA’s SNOTEL system provide long, continuous records of snow depth underpinning water resource planning, avalanche forecasting and climate research across the western United States. These records are not clean: ultrasonic depth sensors are prone to various unwanted occurrences, such as false echoes and battery-related faults. This project set out to build and evaluate a small pipeline for detecting and correcting this noise in a single SNOTEL station’s daily snow depth record, using a manually labelled sample of 241 days as ground truth.

The work proceeds in five stages: acquiring the true, uncorrected sensor readings for a single SNOTEL station; manually labelling a stratified sample of days as good or noise, informed by cross-referencing water equivalent and air temperature; implementing a simple rolling median (Hampel) baseline filter; building three machine learning models of increasing complexity (an ARIMA-residual detector, an Isolation Forest, and an LSTM autoencoder); and producing a denoised version of the dataset using whichever method performed best under rigorous, paired statistical testing. None of the three machine learning approaches could be shown to significantly outperform the simple statistical baseline. In one case the baseline was shown significantly better, even after the machine learning model had been substantially improved. Recent published work in the same domain achieved considerably stronger machine learning performance, but only at a data scale and sensor richness this project did not have access to. Data availability, not a failure of the methods themselves, is the more defensible explanation. The dissertation sets out the debugging process and the statistical methodology behind that conclusion in some detail, alongside each method’s practical strengths and weaknesses.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims and objectives	1
1.3	Relevance to future work	2
1.4	Dissertation structure	2
2	Literature Review	3
2.1	The general problem	3
2.2	Imputation techniques	4
2.3	Model-based approaches: ARIMA and state-space methods	5
2.4	Machine learning approaches	5
2.5	Statistical evaluation of competing methods	7
3	Methodology	7
3.1	Data source	7
3.2	Preprocessing	8
3.3	Manual labelling	10
3.4	Baseline: the rolling median (Hampel) filter	11
3.5	ARIMA-residual model	12
3.6	Isolation Forest	13
3.7	LSTM autoencoder	14
3.8	Feature engineering and the systematic feature ablation study	16
3.9	Statistical comparison methodology	17
3.10	Producing the final denoised dataset	17
4	Results: Comprehensive Evaluation Against the Labelled Test Set	18
4.1	Headline results	18
4.2	What the paired comparison shows	19
4.3	Where the methods differ	20
4.4	The effect of pooling more training data	21
4.5	Practical performance on concrete cases	21
4.6	Data quality improvement in practical terms	23
4.7	A methodological limitation	24
5	Strengths and Weaknesses of Each Method	24
6	Discussion	26

7	Future Work	27
8	Self-Assessment	28
9	How to Use This Project	29
9.1	Setting up the environment	29
9.2	Running the pipeline	31
	Bibliography	31

1 Introduction

1.1 Motivation

Snow depth data from remote automated stations is used far beyond the meteorological community that collects it: water utilities forecast spring runoff from it and avalanche forecasters judge snowpack loading, while hydrologists track decades of it for signs of a shifting climate. Serreze et al. (1999) [19], in the foundational study establishing SNOTEL as a serious climatological resource, found SNOTEL’s snow water equivalent readings compared favourably with independently collected snow course measurements, but only after developing dedicated quality-control procedures to screen outliers first, a reminder that the tension between SNOTEL’s long-term value and the work needed to trust its raw output has existed since close to the network’s start. All of this depends on a number labelled “snow depth, 47 inches, 14 March” actually meaning something close to that. In practice, the sensors are simple ultrasonic rangefinders mounted above the snow surface. They are vulnerable to a range of physical error sources, such as wind vibrating the mounting structure or distorting the return pulse, animals disturbing the snow beneath the sensor, ice obstructing the transducer, a poorly reflective target surface, and general equipment malfunction. Ryan, Doesken and Fassnacht (2008) [17], in a field evaluation of the same class of sensor used across the SNOTEL network, documented this same set of error sources from manufacturer fault reports and their own multi-site testing. That grounds the list in an established characteristic of the technology, rather than a guess specific to this project’s station. The result is a record mixing real snow accumulation and melt with sporadic sensor error.

The project grew out of a fairly ordinary question. Given eight years of daily snow depth readings from a real SNOTEL station, noise and all, can a machine learning model tell a real snow event from a sensor fault more reliably than a simple statistical rule? If it can, there is a case for deploying such models operationally to improve a dataset many other people already rely on. If it cannot, that is worth knowing too, and worth understanding why.

1.2 Aims and objectives

The aims of this project were:

1. To acquire a real-world snow depth dataset.

2. To build a small ground-truth test set by manually labelling data as good or noise.
3. To implement a simple statistical baseline filter and establish how well it performs against that ground truth.
4. To implement at least two machine learning models capable of detecting the same noise, and compare their performance against the baseline using proper statistical methodology.
5. To use whichever method performed best to produce a final denoised dataset.
6. To reflect on what the results say about machine learning’s practical value for this applied problem.

1.3 Relevance to future work

Beyond the specific outcome, this project gave hands-on experience with things that come up repeatedly in applied data science and are easy to underestimate from outside it. Evaluating machine learning models properly, with attention to class imbalance, statistical significance and overfitting to a small labelled set, is a different skill from simply training a model and reporting its accuracy. All of this bears on a career in data science or environmental data analysis, where interrogating a dataset critically before trusting any model built on it matters greatly.

1.4 Dissertation structure

Section 2 reviews the literature on time series smoothing, imputation and outlier filtering. Section 3 describes the methodology in full: data acquisition, preprocessing, manual labelling, and the design of each detection method. Section 4 presents the evaluation against the labelled test set. Section 5 analyses each method’s strengths and weaknesses. Section 6 discusses the results against published work. Section 7 outlines future work. Section 8 is a self-assessment. Section 9 explains how to run the code.

2 Literature Review

2.1 The general problem

Environmental and industrial sensor time series share a common set of problems regardless of the specific domain. Common occurrences are missing observations, isolated erroneous readings, and slight errors with a sensor’s true calibration. The literature broadly separates the resulting techniques into three overlapping categories, smoothing, imputation, and outlier detection, although in practice a single method (as this project shows) often does more than one of these jobs at once.

Classical smoothing methods aim to separate an underlying signal from noise superimposed on it. The simplest, the moving average, replaces each point with the mean of a surrounding window. It is easy to implement, but not robust: a single extreme outlier pulls the average toward it, which is the opposite of what a filter should do when the goal is to catch that outlier. The moving median improves on this, since an outlier has to make up more than half the window before it can move the median at all. Extending a moving median with a formal rule for how far a point can deviate from its local median before being flagged gives the Hampel identifier. It uses the median absolute deviation, scaled by approximately 1.4826 to make it comparable to a standard deviation under a normal distribution, as the measure of “typical” local spread, and flags a point when its deviation exceeds some multiple of that, commonly three. Leys et al. (2013) [10] make the case for this spread measure well: the mean and the standard deviation are themselves highly sensitive to the very outliers a filter is trying to find, whereas the median and median absolute deviation each have a breakdown point of 50 percent, which is a considerably more robust property for the kind of contaminated field data this project works with. A robust local centre paired with a robust local spread is what this project implements as its baseline, a long-established technique from the robust statistics literature. Roos-Hoefgeest Toribio et al. (2025) [16] confirm that implementing a Hampel filter correctly remains an active concern beyond the textbook formula, though from a different angle than this project’s own zero-spread edge case in Section 3.4: they show the standard median absolute deviation estimator becomes computationally expensive at scale, and propose a faster estimator that preserves the same statistical robustness.

More recent work extends this basic idea further. STL (Seasonal-Trend decomposition using Loess), given an accessible modern treatment in Hyndman and Athanasopoulos’s textbook [8], separates a series into trend, sea-

sonal component and residual, treating large residuals as candidate anomalies. RobustSTL (Wen et al., 2019) [24] extends this to resist the sharp, isolated outliers that would otherwise distort the decomposition, an important property for a series like snow depth with both a strong annual shape and occasional sharp, real storm events. Functional data analysis, fitting each season’s curve with splines rather than discrete points, offers a related way of separating expected shape from local deviation.

2.2 Imputation techniques

Once a reading is identified as noise, or is simply absent, the second problem is deciding what value should stand in its place. The simplest approach, forward-filling the last known value, is easy but produces an artificially flat series across the gap, actively misleading for a quantity like snow depth that changes continuously. Linear interpolation, drawing a straight line between the values either side of a gap, is a marginal improvement suited to short gaps, but the straight-line assumption becomes progressively less defensible the longer the gap runs; this project relied on it only briefly, to give the ARIMA model a continuous series to fit against, never as the actual reported output.

More statistically grounded methods generally fall into two families. Model-based imputation, of which the Kalman smoothing approach discussed below is one example, treats missing values as unobserved states of a model fitted to the surrounding data, producing an estimate consistent with that model rather than a straight line. Multiple imputation, common in statistics more broadly, generates several plausible values per gap instead of committing to one estimate, to represent the uncertainty properly. This project did not use it: a single, clearly-flagged best estimate was judged sufficient, though multiple imputation would be a more statistically complete alternative a future extension could adopt, particularly for the longest gaps described in Section 3.10.

This project’s own final imputation step, described in Section 3.10, sits closest to the first family. It fills both flagged noise and readings that were simply absent with a local, non-parametric estimate: the same rolling median already used for detection, with a wider second pass for gaps too long for the first to reach. That was a deliberate choice, keeping imputation as consistent and interpretable as the detection method that identified which points needed replacing in the first place.

2.3 Model-based approaches: ARIMA and state-space methods

A different family of techniques models a time series explicitly, using the fitted model’s own prediction error as the anomaly signal rather than a fixed local window. ARIMA (autoregressive integrated moving average), given a thorough modern treatment by Hyndman and Athanasopoulos [8], predicts each value from recent past values, the differenced series, and recent forecast errors. A large gap between prediction and observation can then serve as an anomaly score, much as a Hampel filter’s deviation score does. This project implements such a model, and Section 3 turns up a counterintuitive finding along the way: a naive, globally-fitted ARIMA model, because its parameters minimise error across the whole training series, ends up partially learning to anticipate the very large, recurring spikes it is meant to flag. That is a fairly well-known tension in unsupervised anomaly detection more broadly, since a model trained on data already containing the anomalies of interest will, to some degree, treat them as normal.

State-space methods, most notably the Kalman filter, offer a further alternative that separates a hidden “true” state from noisy observations of it. They are typically applied to gap-fill spatially dense sensor networks, where nearby sensors inform each other spatially as well as temporally. SNO-TEL stations are usually tens of kilometres apart rather than a dense mesh, so a spatially-dependent approach would not transfer to this dissertation’s single-station, temporal-only problem without a fundamentally different data source.

2.4 Machine learning approaches

Anomaly detection using machine learning generally falls into two broad families relevant to this project: tree-based ensemble methods and deep learning sequence models.

The Isolation Forest (Liu, Ting and Zhou, 2008) [11] is an unsupervised tree-based method that isolates points using randomly chosen features and split values, on the reasoning that an anomalous point needs fewer random splits to separate from the rest of the data than a typical point does. It suits this problem because it can take several variables at once, learning relationships between them (whether snow depth and water equivalent move together, say) rather than judging one variable in isolation as the baseline and ARIMA both do. A distinction from Chandola, Banerjee and Kumar’s (2009) [5] widely cited survey matters here: a point anomaly is unusual across

the whole dataset, while a contextual anomaly is only unusual relative to its immediate surroundings. Experimentation in Section 3 found that feeding an Isolation Forest raw values causes it to learn point anomalies rather than the contextual ones of actual interest, since a near-zero snow-free-season reading is one of the most common value combinations across the record and gets treated as maximally normal by a method built to find rare combinations.

Autoencoders, specifically LSTM-based sequence autoencoders (building on the long short-term memory architecture of Hochreiter and Schmidhuber, 1997 [7]), take a different approach: trained to reconstruct its own input through a compressed representation, an autoencoder learns to reconstruct common “normal” patterns well and struggle with rare, anomalous ones, and the reconstruction error, not the network’s prediction, becomes the anomaly score. This specific architecture, an LSTM encoder-decoder scored on reconstruction error rather than trained against any label, follows the EncDec-AD approach of Malhotra et al. (2016) [12], who demonstrated it across several real-world sensor datasets, detecting anomalies in predictable, unpredictable, periodic and aperiodic series alike without labelled training, the same setup this project’s LSTM autoencoder in Section 3.7 uses. An LSTM autoencoder can in principle learn much richer temporal and cross-variable relationships than the Hampel filter or Isolation Forest, at the cost of requiring considerably more data and careful training.

The closest published work found in this project’s literature search is Svoboda et al. (2025) [20], who developed a deep learning classifier, CleanSnow, for automated quality control of snow height time series from a network of Swiss stations. Their temporal convolutional network was trained on roughly 2.1 million labelled samples, subsampled from about 7 million collected across 20 stations, 14 for training and 6 held out for testing. Several stations carried additional sensors, snow surface temperature and reflected shortwave radiation, not present at the station used here; the final model used snow height, snow surface temperature, air temperature and reflected shortwave radiation as its four input features, reporting close to 98 percent accuracy and an F1 score of 97.8 percent, considerably higher than anything achieved in this dissertation. This comparison is returned to in detail in Section 6, since it is central to how this project’s own, more modest, results should be interpreted.

A second, closely related study is Blandini et al. (2023) [4], who trained a random forest classifier to detect random errors and separate genuine snow readings from bare-ground or vegetation readings, using ultrasonic sensors across Italy’s Aosta Valley. Trained on around 1.9 million measurements from 43 sensors, their model reached F1=0.89 for random-error detection

and 0.99 for the snow-versus-bare-ground distinction. The detail most relevant here is what happened when they evaluated the same model against 27 different sensors elsewhere in Italy it had not been trained on: F1 for random-error detection fell below 0.05 across every year tested, dropping sharply despite nearly two million training measurements. This result is returned to in Section 6, since it corroborates, at far larger scale than this project could test, this dissertation’s own multi-station pooling finding in Section 3.7: a noise-detection model trained at one set of stations does not reliably transfer to another, even trained on far more data.

2.5 Statistical evaluation of competing methods

Comparing two models’ aggregate F1 side by side is weaker evidence than it looks, a point emphasised less often in applied machine learning writing than it deserves. Two models can achieve identical scores while disagreeing substantially on individual cases, and with a small evaluation set the difference between two aggregate scores can easily be ordinary sampling variation rather than a real difference in ability. McNemar’s test, given an accessible modern treatment by Raschka (2018) [15], is a standard paired significance test built for this purpose: comparing two classifiers on the same test cases by looking specifically at where they disagree. This project used it at several points, but also ran into a limitation of it: because McNemar’s test treats every correct classification equally regardless of class, it can flag a difference as significant even when F1 for the minority “noise” class has moved the other way. Once that was discovered, a paired bootstrap test [3], resampling the labelled days and recomputing F1 on each sample, was used instead, documented in Section 3 as part of the methodology rather than left out.

3 Methodology

3.1 Data source

The dataset used throughout this project is the daily snow depth, snow water equivalent, air temperature, and precipitation record from Csx Lab, station 428:CA:SNL in the USDA Natural Resources Conservation Service’s SNOTEL network [21], located at 6,890 feet in Nevada County, California, with data drawn from 1 October 2015 to 30 September 2023, a span of 2,922 days. Data was retrieved from the network’s public AWDB REST API.

Csx Lab is not an arbitrary station among SNOTEL’s several hundred sites. The SNOTEL sensor is co-located with UC Berkeley’s Central Sierra

Snow Laboratory (CSSL), built in 1946 by the U.S. Weather Bureau and Army Corps of Engineers (University of California, Berkeley, Office of the Vice Chancellor for Research, n.d.) [23]. CSSL’s own website states it maintains the longest continuous snowfall and snow depth climatological record in the world, dating to 1879 (University of California, Berkeley, Central Sierra Snow Laboratory, n.d.) [22]. A Berkeley News profile quoted on the university’s research-unit page separately describes one CSSL scientist’s own redundant backyard setup of four precipitation instruments cross-checking each other, an unusually close, hands-on measurement culture rather than a formal network-wide CSSL claim (University of California, Berkeley, Office of the Vice Chancellor for Research, n.d.) [23]. None of this makes the sensor immune to the noise this dissertation studies; the raw data confirms it is not. But the station sits within a well-supported research context, with active SNOTEL records since 1980 (confirmed on NRCS’s station page) to support the eight-year window comfortably. That, together with the SNOTEL network’s general legitimacy as described by Serreze et al. (1999) [19], gives citable grounds for reporting on it in depth.

The raw, uncorrected sensor readings were retrieved using the API’s `returnFlags` and `returnOriginalValues` parameters, which return both the true sensor reading and NRCS’s quality flag for it alongside the edited “official” value. Comparing the two shows 39.5 percent of all snow depth readings in the raw record differ from NRCS’s official value, including isolated raw readings of over 360 inches (more than thirty feet) corrected down to zero. This raw record, not the official one, is the dataset used throughout. The official value is retained alongside it purely as a reference and is never shown to the manual labeller or used as ground truth.

3.2 Preprocessing

Once the true raw readings were retrieved, several categories of physically impossible values needed handling before any detection method could reasonably operate. Negative snow depth and water equivalent readings cannot occur physically, so these were converted to missing values: 451 such readings turned up in the snow depth record. A second fault, isolated readings far above anything ever officially recorded, such as the 360-inch false echoes described above, was handled with a ceiling set relative to each station’s own highest validated historical depth, multiplied by 1.25 to leave margin for a real, if unprecedented, storm; 277 such readings were removed at Cssl Lab. This ceiling is deliberately data-driven and station-specific rather than a fixed number, and the 1.25 multiplier is not robust across an arbitrarily

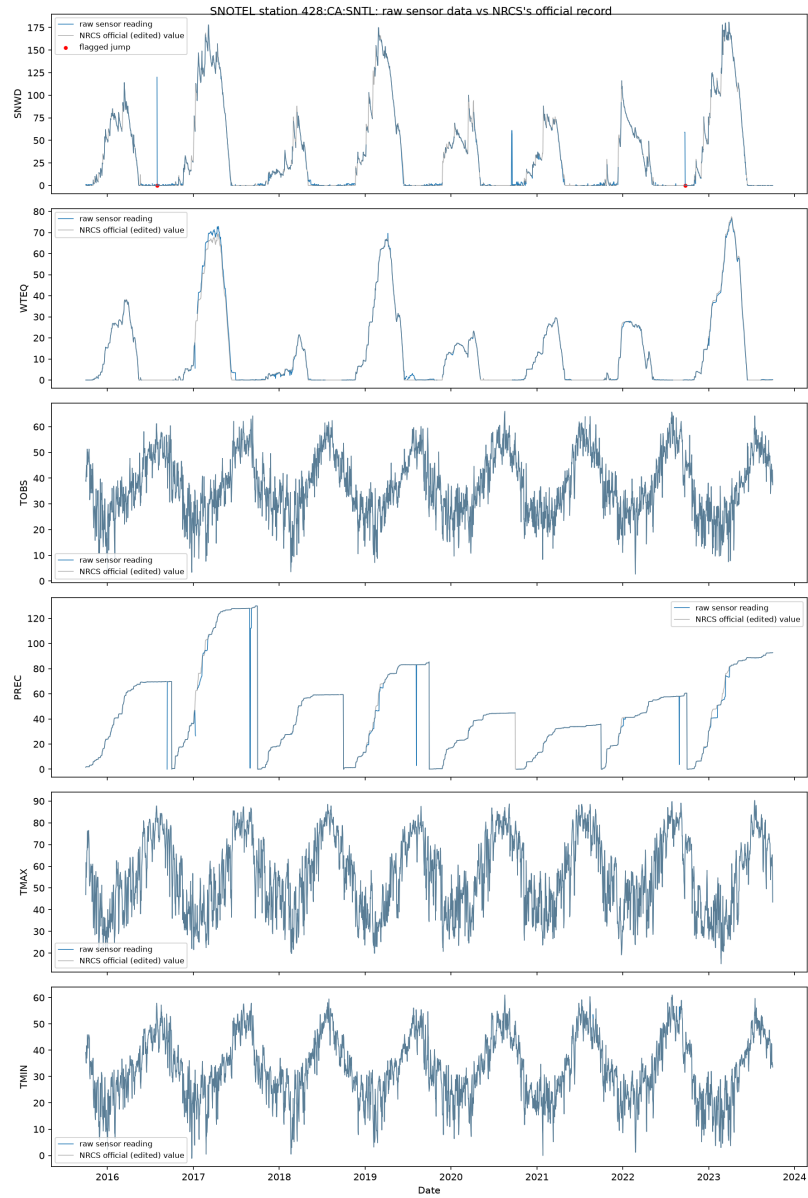


Figure 1: The raw sensor reading against NRCS's official (edited) value for all six variables at Css Lab, with jumps flagged during preprocessing marked on the snow depth panel.

wide range: raising it only to 1.5 already lets a cluster of 24 readings back through as “plausible.” Every one of those 24, though, reports the identical value of 259 inches on unrelated dates, with official snow depth ranging from 12 to 154 inches and water equivalent from 1.7 to 70.3 inches, uncorrelated with a fixed 259-inch snowpack but consistent with real conditions, the signature of a sensor stuck on one erroneous value. The chosen 1.25 sits safely below the roughly 1.43 point where this cluster would start being missed, so while not derived from a formal statistical test, it was empirically verified against the data it is meant to exclude.

A day-to-day jump detector was also implemented, flagging any single-day snow-depth change larger than a robust, data-driven threshold, to surface candidate sensor faults for manual review. An early version had a subtle bug: because the threshold was a percentile of day-to-day changes across the whole dataset, and the large false-echo spikes had not yet been removed when it was first computed, those spikes inflated the threshold so much it stopped catching anything, including the spikes that caused the problem. Reordering the pipeline so physically impossible values were removed first resolved this, one of several examples throughout the project where a plausible first implementation produced silently wrong results, discussed collectively in Section 8.

3.3 Manual labelling

A stratified rather than purely random sample of 241 days was selected for manual labelling. A simple random sample of an eight-year record would be almost entirely uneventful “good” days, nowhere near enough noise examples to evaluate anything against. The sample combined five selection rules: days NRCS’s own quality control flagged as suspect on the raw reading (9), days the project’s own jump detector flagged (2), midpoints of runs of five or more identical consecutive readings, indicating either a calm period or a frozen sensor (37), a random sample of days with snow present (150), and a random sample from the whole record regardless of season (50), with duplicates removed. This gave 241 candidate days, 177 labelled good and 64 labelled noise, a noise rate of 26.6 percent.

Each candidate day was judged using a small interactive tool showing snow depth, water equivalent and temperature for a fifteen-day window either side, without revealing NRCS’s own official value or quality flag, so the judgement remained independent of NRCS’s process and could be compared against it afterward. A reading was judged good if it fit smoothly into the surrounding trend and, where a jump occurred, water equivalent moved

consistently alongside it, since two independently measured variables moving together is strong evidence of a real event rather than a single-sensor fault. A reading was judged noise if it was an isolated spike that reverted completely the next day with no corroborating water-equivalent change, or if it sat within a persistent low-magnitude flicker during a snow-free season with water equivalent essentially flat throughout. Some cases were just genuinely hard to call, most notably brief flat winter periods where both variables held an identical value for several days, which could mean either a real calm spell or a telemetry fault repeating the last reading. Where the evidence did not clearly favour one explanation, these days were treated with lower confidence rather than forced into either category.

3.4 Baseline: the rolling median (Hampel) filter

The baseline method follows the Hampel identifier described in Section 2.1. For each day, a rolling median of a centred seven-day window is the “expected” value, and the window’s median absolute deviation, scaled by 1.4826, measures typical local spread. A day is flagged as noise if its deviation from the rolling median exceeds three times this scaled spread, a threshold chosen up front as a standard default rather than tuned against the labelled data, so the comparison against more complex methods would be fair.

Testing turned up a real implementation problem, worth describing because it materially changed the result and illustrates a general point about getting even simple statistical methods right. The median absolute deviation of a seven-day window comes out to exactly zero whenever a majority of the seven values agree, which happens constantly here, since snow depth is recorded in whole inches with long flat stretches during the snow-free season. The first version of the code treated a zero spread as meaning no score could be computed, defaulting such days to “not noise.” That is the wrong call: a day disagreeing with an otherwise perfectly uniform neighbourhood is the single most suspicious case this method can encounter, not one that should be unmeasurable. Fixing this (treating a nonzero deviation against zero local spread as maximally suspicious rather than unknown) changed the baseline’s recall from 0.05 to 0.27 and precision from 0.38 to 0.53, far too large a swing to be a rounding difference. Textbook Hampel-filter accounts generally assume continuous data where an exact tie is rare, so this fix isn’t standard; it is flagged here as a reasoned addition specific to this dataset’s discrete, whole-inch measurements.

3.5 ARIMA-residual model

The second method fits a single ARIMA model, implemented using the statsmodels Python package (Seabold and Perktold, 2010) [18], to the full eight-year series, using its in-sample one-step-ahead predictions as the “expected” value in the same deviation-scoring framework as the baseline. Unlike the baseline’s three-times-spread threshold, there is no citable convention for an ARIMA order. Rather than picking one by feel, the order was selected by an AIC-based grid search fit to the raw series, following standard practice for automatic order selection [8]. That search never looks at the manual labels, so unlike the baseline’s threshold, it carries no risk of being tuned toward a better-looking result. An initial, narrower search picked an order sitting exactly at the edge of the range tried. Widening the range revealed why that edge kept moving rather than settling: almost every strong candidate had a combined AR-plus-MA order of eight, nine, or even ten, with AIC still improving and no sign of levelling off. That is a classic overfitting signature: the extra terms are partly fitting the recurring false-echo spikes themselves, rather than any real structure. The standard fix, used by automatic order-selection algorithms including the Hyndman-Khandakar algorithm described in Hyndman and Athanasopoulos (2021) [8], is to cap the combined order, five being the default in R’s `forecast::auto.arima` implementation of that algorithm; applying that established convention rather than an arbitrary cutoff, the order actually used is (5, 1, 0), the strongest AIC score satisfying that cap. Small gaps were filled by straight-line interpolation purely so the model could be fitted continuously; every comparison against its predictions used the true, ungapped raw readings.

Investigating why this model underperformed the baseline, described fully in Section 4, turned up a useful finding. On the clearest test case, an isolated 120-inch spike surrounded by zero readings, the model predicted 92.16 inches, far closer to the spike than to the zero the surrounding days made obvious. This happens because the model’s parameters minimise error across the entire training series, which contains several recurring examples of this same large, isolated spike; a model whose whole objective is reducing overall prediction error will partially learn to anticipate a pattern that recurs often enough, even if that pattern is the noise it should flag. The baseline’s local, non-parametric median has no equivalent weakness, recomputed independently each window with no memory of anomalies elsewhere. A SARIMAX version using water equivalent as an exogenous variable was also tested; it improved precision but left recall essentially unchanged, so access to the extra variable did not resolve the underlying issue.

3.6 Isolation Forest

The third method is a scikit-learn (Pedregosa et al., 2011) [14] Isolation Forest, an unsupervised, multivariate, tree-based anomaly detector. An initial version, fed raw snow depth, water equivalent, temperature and precipitation, performed at essentially zero recall regardless of the contamination setting tested. Investigating this failure led back to the point-versus-contextual anomaly distinction from Section 2.4. Roughly half of any year here has no snow at all, so a near-zero reading is one of the most common value combinations in the record, and a method built to find globally rare combinations treats it as maximally normal rather than flagging it, while a legitimate deep-winter reading looks anomalous purely for being large. Replacing the four raw variables with each one’s own local deviation score, using the same rolling-median transform as the baseline, reframed the problem from “is this globally rare” to “is this inconsistent with what was happening immediately around it,” which is what the model actually needed to detect. This single change took recall from 0.00 to 0.25.

Two further refinements were made, documented as evidence-based additions rather than arbitrary tuning. A feature representing the gap between the snow depth and water equivalent deviation scores was added, encoding the cross-referencing logic used throughout manual labelling: a large gap, where snow depth looks unusual but water equivalent does not back it up, is what the labelling protocol treats as most suspicious. This improved F1 from 0.26 to 0.32. A daily minimum battery voltage feature came next, added after testing showed a significant voltage difference between good and noise days surviving within-month checks, ruling out season; unlike several rejected candidates in Section 3.8, this one had a physically plausible causal story behind it. There was also a stability problem to solve: a single Isolation Forest fit was noticeably unstable even with the final feature set, F1 varying between 0.33 and 0.36 across five random seeds, so the final version fits fifteen independently seeded forests and takes a majority vote, the same averaging-over-randomness principle the algorithm itself is built on, one level up. Fifteen was not just a round number: fitting independent replicates at a range of sizes shows the spread in F1 shrinking from a standard deviation of 0.010 at five seeds to 0.004 at fifteen, only reaching 0.002 by twenty, past the point of diminishing returns. The final, tuned Isolation Forest reached an aggregate F1 of 0.35, matching the baseline.

3.7 LSTM autoencoder

The fourth method is a small LSTM autoencoder, implemented using TensorFlow and its Keras interface (Abadi et al., 2016) [1], taking fourteen-day sliding windows of snow depth, water equivalent, temperature and a same-day precipitation feature, encoding each window through a single LSTM layer of eight hidden units and reconstructing it through a matching decoder. The architecture, an LSTM encoder feeding a repeated-vector layer feeding a second LSTM decoder, follows the standard Keras pattern for sequence autoencoders set out by Keras’s own creator (Chollet, 2016) [6], rather than an arrangement invented from scratch, so a reader unfamiliar with this library can see how the description above was implemented in code, not only why it works conceptually. The model is trained unsupervised, without seeing the manual labels, to reconstruct the raw series; each day’s reconstruction error, converted into the same local-deviation score used throughout, becomes its anomaly score. An epoch sweep found the model meaningfully undertrained at low epoch counts, F1 improving from 0.16 at 20 epochs to 0.19 at 50 to 0.26 at 100, then falling to 0.22 at 150 and 0.21 at 200 as it began overfitting; 100 epochs, the clear peak, was used (see Figure 2).

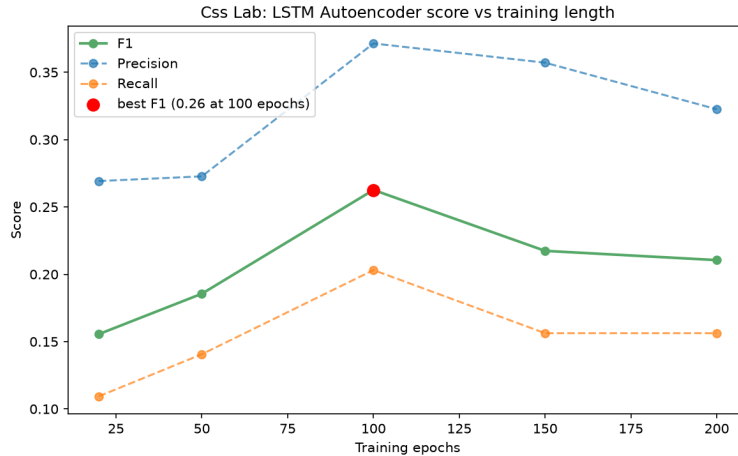


Figure 2: LSTM autoencoder F1 score as a function of training epoch count, showing a clear peak at 100 epochs before the model begins to overfit.

Data from five further California SNOTEL stations was acquired and processed through the same pipeline as Css Lab, but without manual labels of their own; they contribute unlabelled training data only, while Css Lab

remains the sole ground truth throughout. They were also selected reproducibly rather than picked informally: the three active California SNOTEL stations with the highest elevation, excluding Coss Lab, went in automatically, and two further stations were drawn at random, with a fixed seed, from the remaining pool. That mixes stations chosen for being climatically most different from Coss Lab with stations chosen with no selection bias at all, rather than an ad hoc shortlist.

Table 1: The primary evaluation station and the five stations used for pooling.

Station	Triplet	Elevation (ft)	Role
Coss Lab	428:CA:SNTL	6,890	Primary; manually labelled evaluation station
Leavitt Lake	574:CA:SNTL	9,600	Pooling only; top-elevation selection
Virginia Lakes Ridge	846:CA:SNTL	9,400	Pooling only; top-elevation selection
Summit Meadow	1052:CA:SNTL	9,360	Pooling only; top-elevation selection
Ebbetts Pass	462:CA:SNTL	8,660	Pooling only; random selection
Ward Creek #3	848:CA:SNTL	6,750	Pooling only; random selection

Training on two stations’ combined data made the LSTM’s F1 slightly worse, from 0.26 to 0.20, and all six stations made it worse again, to 0.17. The same test for the Isolation Forest produced a sharper, non-monotonic pattern: F1 fell from 0.35 to 0.17 at two stations before partially recovering to 0.28 with all six, both trends shown in Figure 3. The most plausible explanation, discussed further in Section 4.4, is that different SNOTEL stations have somewhat different noise characteristics, and pooling them dilutes rather than reinforces what either model learns about “normal” at the one station being evaluated.

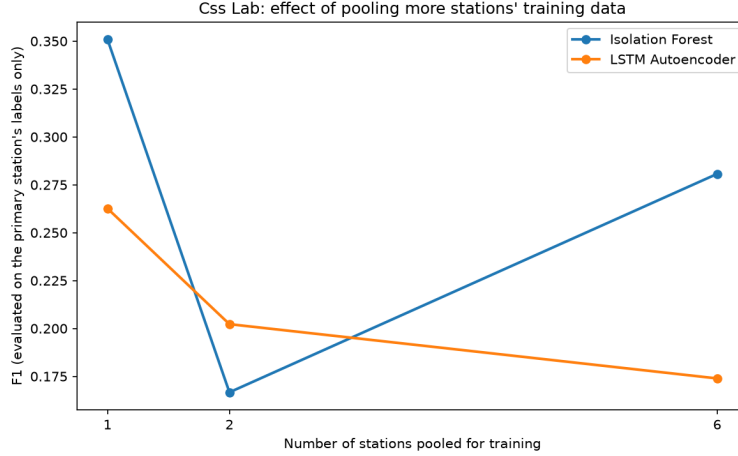


Figure 3: The effect of pooling additional stations’ unlabelled data on LSTM autoencoder and Isolation Forest F1, evaluated at 1, 2 and 6 pooled stations against the same Css Lab labelled test set.

3.8 Feature engineering and the systematic feature ablation study

Beyond the variables described above, this project also fetched and tested battery voltage, soil moisture and soil temperature at three depths, and daily maximum and minimum air temperature, to establish rigorously which were worth including rather than adding them on impression. Soil moisture and temperature at all three depths initially appeared highly significant between labelled good and noise days, p -values below 0.001 throughout. This was investigated further before being trusted, since labelled noise days cluster heavily in the shoulder seasons while good days cluster toward midwinter, and both variables have a strong seasonal cycle independent of any sensor fault. Comparing good and noise days within the same calendar month rather than pooled across the year showed almost all of this apparent significance disappeared, confirming it was overwhelmingly a seasonal confound. Battery voltage, tested the same way, retained a significant difference within individual months, which is why it alone was carried forward.

This conclusion was re-tested formally with a systematic ablation study: each candidate feature was added to the existing Isolation Forest feature set in turn, its effect measured with a paired bootstrap significance test on F1, run on the same evaluable days for a fair comparison. An earlier version used

McNemar’s test instead and produced a contradictory result: one feature was flagged as a significant improvement despite its F1 having fallen. The reason is that McNemar’s test weighs both classes equally while F1 is weighted toward the minority noise class, so the two can disagree on an imbalanced dataset. Once corrected, the ablation study found none of soil moisture, soil temperature, or daily max/min temperature produced a significant improvement, and two of the three soil moisture depths produced a significant reduction, giving the final six-feature Isolation Forest an evidence-based justification rather than one built on intuition.

3.9 Statistical comparison methodology

Two further statistical tools were used to compare the four methods properly. Wilson score confidence intervals, given a modern, freely accessible treatment by O’Neill (2021) [13], were computed for each method’s recall on the labelled test set, chosen deliberately over the simpler normal approximation interval. That simpler interval is known to behave poorly on small, moderately extreme proportions like this project’s 64-example noise class, sometimes producing bounds outside zero-to-one or badly miscalibrated coverage; the Wilson interval stays well-behaved under these conditions. These intervals overlapped substantially across most method pairs, meaning that with only 64 noise examples, an aggregate score alone cannot confidently show one method is better than another. A paired McNemar test, comparing two methods on the days where they disagree rather than treating scores as independent, gave a more discriminating answer for the strongest comparison available: even after every improvement in Section 3.6, the baseline was correct significantly more often than the Isolation Forest on days where the two disagreed, 23 against 7, unlikely to arise by chance. This distinction between an aggregate score and a properly paired comparison is returned to as a central methodological point in Section 4.

3.10 Producing the final denoised dataset

The rolling median baseline was used to produce a final, complete, denoised version of the dataset, since it was the only method to survive every statistical comparison performed. Every day originally missing due to a physically impossible reading, and every day flagged as noise by the filter, was replaced with the filter’s own rolling median estimate. An initial version left 365 days, 12.5 percent of the record, still unfilled, since the seven-day detection window cannot bridge a gap longer than a few days. Investigating showed the longest

gaps sat consistently within snow-free summer stretches, readings immediately either side already at or near zero, justifying a second imputation pass with a wider thirty-one day window for whatever the first pass could not reach. This reduced the unfilled total to 62 days, 2.1 percent, concentrated in the driest parts of summer. Rather than widen the window indefinitely to chase this residual, which would eventually stop being informed imputation and become an unfounded assumption, these days were simply left missing and flagged as such, alongside a column recording which pass, if either, produced each imputed value.

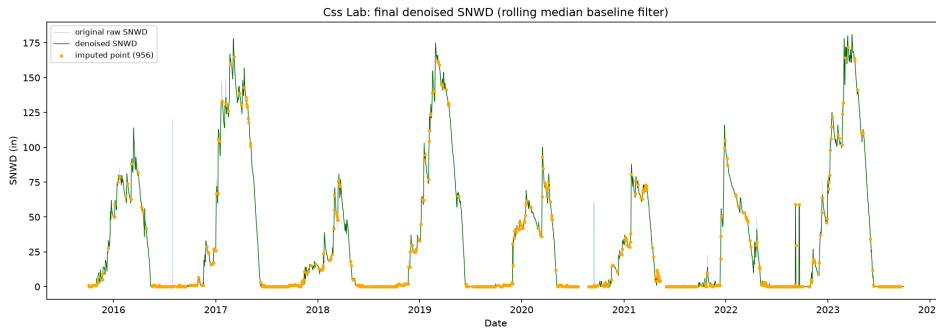


Figure 4: The final denoised snow depth series against the original raw series, with every imputed point marked.

4 Results: Comprehensive Evaluation Against the Labelled Test Set

4.1 Headline results

All four methods were evaluated against the same 241 manually labelled days (177 good, 64 noise), using identical precision, recall and F1 definitions, with the noise class treated as positive throughout since correctly identifying noise, not good data, is the practically important outcome here. This reflects a well-established point in the class imbalance literature: with noise only 26.6 percent of the labelled set, a method could reach misleadingly high accuracy simply by predicting “good” for almost everything, the risk that motivates the imbalanced-learn toolbox described by Lemaitre, Nogueira and Aridas (2017) [9]. F1, by construction, gives no credit for correctly predicting the majority class, which is why it, not accuracy, is the headline metric throughout this dissertation.

Table 2: Headline evaluation results for all four methods against the 241-day labelled test set.

Method	Accuracy	Precision	Recall	F1
Rolling median baseline (Hampel filter)	0.74	0.53	0.27	0.35
ARIMA-residual	0.74	0.52	0.17	0.26
Isolation Forest (final, six features, ensembled)	n/a	0.35	0.35	0.35
LSTM autoencoder	0.70	0.37	0.20	0.26

The same comparison is shown visually in Figure 5. Read either as table or chart, the result looks almost anticlimactic: the simplest method and the most heavily engineered machine learning method arrive at the same F1 score, and the other two score lower still. What the chart makes clear, in a way the F1 column alone does not, is that the baseline and the Isolation Forest reach that identical F1 by very different routes; the baseline through higher precision and lower recall, the Isolation Forest with the two almost exactly balanced. That is a hint the two are making different kinds of mistakes rather than converging on the same behaviour, and it is precisely why the paired statistical testing in Section 3.9 mattered: two methods can arrive at an identical aggregate score while genuinely differing in ability, or while not differing at all, and only a test that looks at where they disagree can tell those situations apart.

4.2 What the paired comparison shows

The paired McNemar test between the baseline and the final Isolation Forest, run on days both methods could score, found the baseline correct on 23 days the Isolation Forest got wrong, versus only 7 the other way, unlikely to arise by chance ($p = 0.005$). Despite an identical F1 score, the baseline was shown to be the stronger method once compared properly. Equivalent comparisons between the baseline and the ARIMA model, and the baseline and the LSTM autoencoder, did not reach significance, so this project cannot confidently claim the baseline beats those two, but nor can it claim the reverse; the evidence is simply insufficient to distinguish them, not that they are equivalent.

Taken together, across every comparison attempted (a properly labelled 241-day test set, three machine learning architectures, feature engineering in-

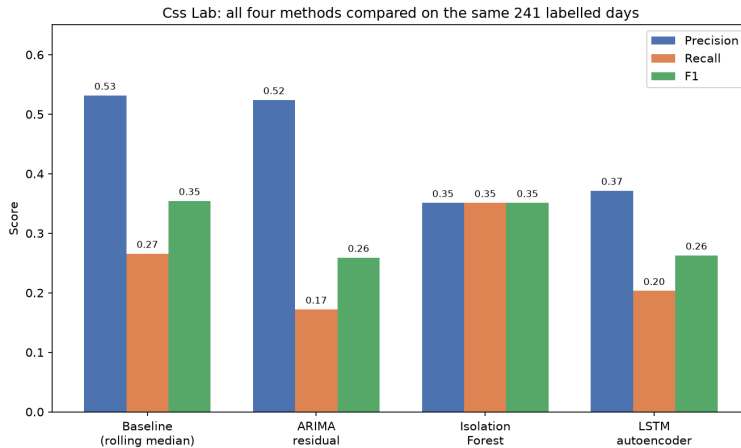


Figure 5: Precision, recall and F1 for all four methods, shown side by side against the 241-day labelled test set.

formed by the manual labelling process, and unlabelled training data pooled from up to five further stations), no machine learning method was shown to significantly outperform a simple, seven-line rolling median filter on this dataset.

4.3 Where the methods differ

Aggregate scores obscure an important qualitative pattern only visible by inspecting individual predictions against the underlying plots. The baseline and ARIMA, both univariate, reliably catch large, isolated single-day spikes surrounded by otherwise stable readings, since these produce an enormous local deviation regardless of other variables. Both, by construction, cannot use water equivalent or temperature at all, and so miss the subtler cases manual labelling relied on cross-referencing to resolve, most notably the persistent low-magnitude flicker during snow-free shoulder seasons, where the day-to-day change is too small to look unusual alone.

The two multivariate methods, Isolation Forest and the LSTM autoencoder, were given the information needed to catch these subtler cases and, to a limited extent, did: the engineered snow depth versus water equivalent mismatch feature was the single largest driver of improvement found anywhere in this project. But inspecting their false positives turned up a limitation. A large share fall on days with large snow depth values during legitimate storms, exactly where snow depth and water equivalent moving to-

gether should have signalled “this is real” rather than “this is noise.” Handing the models the raw material to learn that relationship, as separate deviation features, did not reliably translate into them learning it, at least not at the scale of data available here.

4.4 The effect of pooling more training data

The multi-station pooling experiments described in Section 3.7 produced a clear, if disappointing, result, visible at a glance in Figure 3: adding more unlabelled training data from other stations did not improve either machine learning method, leaving both worse off than the single-station baseline in every configuration tested, though by different routes. The LSTM autoencoder declined steadily as more stations were added, while the Isolation Forest dropped sharply at two stations and then partially recovered at six, never fully returning to its one-station score. The most plausible explanation is that SNOTEL stations at different elevations and microclimates, with presumably different sensor calibration histories, end up with measurably distinct noise characteristics. Leavitt Lake, the highest-elevation pooling station, has proportionally much less data missing outright after preprocessing than Coss Lab (13.3 percent against 24.9 percent), a different noise profile in its own right, not just a scaled version of Coss Lab’s. The proportion of readings NRCS edits at each station, by contrast, is close (41.8 percent against 39.5 percent), so it is specifically the pattern of implausible and missing readings, not the rate of NRCS editing, that differs most. Pooling stations whose noise profiles genuinely differ, with no mechanism for the model to learn that they differ, appears to have diluted rather than reinforced what either model learned about “normal” at the one station this project’s ground truth actually covers.

4.5 Practical performance on concrete cases

Aggregate metrics can make it hard to picture what a model is doing on any single day, so it is worth grounding the numbers above in a small number of specific, representative cases encountered during manual labelling and evaluation.

On 30 July 2016, the raw record shows a short cluster of extreme false-echo readings, 365 inches on both 28 and 29 July followed by 120 inches on 30 July, reverting cleanly to zero on 31 July. The two 365-inch readings are removed by the preprocessing ceiling filter (Section 3.2), leaving 30 July’s 120-inch reading as an isolated value in the working dataset every method

sees, bounded by missing data before it and a clean zero after. Both the baseline and the final Isolation Forest caught this day without difficulty, since the deviation is large enough to stand out under almost any locally-scored rule. ARIMA, characteristically, did not: this is the same day used in Section 3.5 to illustrate the model’s tendency to partially anticipate its own training data’s recurring spikes, predicting 92.16 inches rather than the near-zero value the surrounding days would suggest, a comparatively small deviation score well inside its flagging threshold rather than a near miss.

A harder, more instructive case comes from October 2015, where snow depth flickered between zero and one inch for roughly three weeks while water equivalent sat flat at zero throughout. No single day produces a large deviation alone, and a method looking at snow depth in isolation, like the baseline or ARIMA, has little basis for flagging any day as unusual. This is precisely where the multivariate methods’ access to water equivalent should, in principle, have been decisive: the water equivalent sensor was independently reporting nothing there at all. This is what the engineered mismatch feature was built to capture for the Isolation Forest, and its addition was the single largest driver of improvement in the feature ablation study (Section 3.8), concrete evidence this failure mode was addressed rather than ignored.

A third case worth noting is 22 January 2017, a day labelled good in the middle of a clear, multi-day storm: snow depth rose steadily from 94 inches five days earlier to 147 inches, water equivalent rising alongside it from 31.6 to 41.1 inches, textbook evidence of a real accumulation event. ARIMA correctly left this day alone, seeing only a large but locally consistent change. The Isolation Forest incorrectly flagged it as noise, the same false-positive pattern described in Section 4.3: it had access to the corroborating water-equivalent rise but did not reliably use it. More surprisingly, the baseline flagged this day too, exposing a second, distinct trigger for the zero-spread edge case in Section 3.4. That fix was framed around long, flat, snow-free stretches, but the same mechanism can strike during a smooth, steadily rising storm. Across the seven days centred on 22 January, four of the seven daily deviations from each day’s own local median were exactly zero, because the storm’s day-to-day increase tracked closely enough with the surrounding trend that several consecutive days sat almost exactly on their own rolling median. That drove the window’s median absolute deviation to zero, and the same maximally-suspicious rule that correctly rescues a real fault in an otherwise flat neighbourhood ended up flagging the real 14-inch jump as an outlier instead. It is a real, if infrequent, cost of the zero-spread fix, not something fully resolved, and it illustrates why precision matters as much as recall for a dataset someone needs to trust the “good” days of, not only the

“noise” days it catches.

4.6 Data quality improvement in practical terms

The project’s final deliverable, described in Section 3.10, is a complete, denoised version of the eight-year Coss Lab snow depth record, and its practical value is best judged by what it changes for someone who would use this data downstream, not by the detection metrics alone.

Before cleaning, the record obtained from the SNOTEL API contained 728 physically impossible readings, including isolated values in excess of 360 inches during midsummer, plus a further 228 days independently flagged as inconsistent with their surrounding context by the baseline filter. Left as it was, this data would require any downstream user, a hydrologist building a runoff model, say, or another student reusing the dataset, to either notice and handle these faults themselves or unknowingly build on top of them. The final output resolves 956 of these problem days, 32.7 percent of the full record, replacing each with a locally grounded estimate rather than simply deleting it, which matters for any downstream use expecting a continuous daily series rather than one full of arbitrary gaps. A further 62 days, 2.1 percent, remain genuinely unresolved rather than forced to an unjustified estimate, and are flagged explicitly rather than silently interpolated: a user inspecting the `SNWD_is_imputed` and `SNWD_fill_method` columns can tell an original reading apart from a seven-day-window estimate, a wider thirty-one day fallback estimate, and a day the pipeline was not confident enough to fill at all, instead of facing a single, undifferentiated column of numbers.

This distinction matters because detection performance and practical data quality improvement are not the same question. A method with modest recall, such as the baseline’s 0.27, can still deliver a meaningfully improved dataset if the noise it catches corresponds to the specific, high-impact faults, here the large false echoes, most likely to distort a downstream runoff estimate, even if subtler, lower-magnitude noise remains uncaught. The worked examples in Section 4.5 support exactly this reading: the cases every method reliably caught were the physically implausible ones most damaging if left in the data, while the cases every method struggled with were smaller in magnitude and correspondingly less consequential downstream, even though just as interesting and difficult methodologically.

4.7 A methodological limitation

Every method here is fitted to the full eight-year raw series, and the 241 manually labelled days used to evaluate it are drawn from that same series, not a separate period the model never saw. This is a limitation worth stating plainly, and it is best understood against what this project set out to produce in the first place (Section 3.10): a complete, denoised version of an existing, fixed historical record, not a model deployed prospectively against readings that have not arrived yet. For that goal, in-sample evaluation is a coherent choice rather than an oversight, since the practical question each method needed to answer, “how well does this method clean the record it is applied to,” is exactly what an in-sample comparison measures. A model for live, ongoing monitoring would need a temporal holdout instead, fitting only on data up to some cutoff and evaluating solely against labelled days from after it, to have real evidence its performance holds up on days never encountered before.

Reserving the final two years, a generous holdout by most standards, leaves only 8 of this project’s 64 labelled noise days to score against; a more conventional final-fifth holdout leaves only 2. This project’s existing confidence intervals (Section 3.9), computed across the full 64-example noise class, are already wide enough that an aggregate score alone cannot distinguish most method pairs, so a comparison against 2 or 8 noise examples could not support any statistical conclusion at all. That is confirming evidence for why a holdout was not pursued, rather than the underlying reason, which is simply that this project’s deliverable does not call for one. Even so, the in-sample setup has a real, observed cost, most visible in ARIMA’s own documented tendency (Section 3.5) to partially anticipate the recurring spikes in its own training data. A prospective, temporally-held-out version of this project, with new labelled data from a period held out from the start, is left as future work in Section 7.

5 Strengths and Weaknesses of Each Method

Rolling median baseline (Hampel filter). Its clearest strength is robustness through simplicity: because its “expected value” is recomputed independently and locally for every window, it has no capacity to be misled by patterns learned elsewhere in the dataset, the same weakness that undermined ARIMA. It is fast, needs no training, needs only two interpretable parameters, and its behaviour on any day can be explained in one sentence, underrated when a method’s output must be defended to a non-technical

stakeholder. Its clearest weakness is that it is entirely univariate: it cannot use water equivalent or temperature, so it cannot apply the cross-referencing logic that resolved most of the genuinely difficult labelling cases. Its performance is also sensitive to this dataset’s discrete, whole-inch resolution, the direct cause of the zero-spread bug in Section 3.4; a naive implementation would have quietly underperformed without that being obvious from the code alone.

ARIMA-residual model. Its main strength, in principle, is modelling trend and momentum explicitly, something a flat local median cannot do, which should in theory suit gradually evolving conditions like a melting snow-pack. In practice this project found a specific weakness outweighing that advantage: fitting a single global model on data that already contains the anomalies of interest lets the model partially learn to expect them, working against the goal of flagging them as unusual. This is a limitation of the global-fitting approach used here rather than of ARIMA generally; a version fitted only on already-verified clean data, or refitted periodically on a rolling window, might avoid this, but was outside this project’s scope.

Isolation Forest. Its principal strength is being multivariate while requiring comparatively little data relative to the LSTM autoencoder, the more practical of the two multivariate methods to deploy given the data available here. Careful feature engineering, informed by manual labelling, produced a real, measurable improvement, a useful finding about extracting value from a method like this on a small dataset. Its weaknesses were instructive too. An initial, natural way of feeding it data completely failed for an explainable reason (learning point rather than contextual anomalies). It was also measurably unstable across random seeds, needing ensembling for a trustworthy result. And even at its best, it did not reliably learn the cross-variable relationship that would have let it match, let alone exceed, what a human labeller did by eye.

LSTM autoencoder. Its strength, in principle, is the greatest expressive capacity of any method tested here, able to learn arbitrary, nonlinear relationships across variables and time. In practice, on this dataset, that capacity was never clearly realised: it scored lower than both the baseline and the tuned Isolation Forest and needed a careful epoch sweep to avoid being badly undertrained. Nor did it benefit from pooled training data, the way its data-hungry reputation might suggest. Its most significant weakness here is a mismatch between its appetite for data and what was available: eight years of daily readings from one station is small by deep learning standards, and pooling more stations to compensate made results worse rather than better.

6 Discussion

The central result of this project, that none of three reasonably implemented machine learning methods could be shown to significantly outperform a simple statistical filter, invites an obvious question: is machine learning simply the wrong tool for this kind of problem? The literature in Section 2 suggests a more specific answer than a flat yes or no.

Svoboda et al.’s CleanSnow work [20] (Section 2.4) tackled a closely related problem, automated quality control of snow height records, using a deep learning classifier and achieved considerably stronger results than anything here, F1=97.8 percent against this project’s best of 0.35. The critical difference appears to be scale and sensor richness rather than method: their model trained on roughly 2.1 million labelled samples from a pool of about 7 million across 20 stations, several instrumented with sensors Coss Lab lacks. To avoid overstating the comparison: their own reported baselines, SNOWPACK (physically-based) and MeteoIO (threshold-based), are not machine learning methods, so this project cannot claim even simple methods only work at that scale in their study. What this project’s evidence does support is narrower: its own labelled sample, 241 days from one station, is smaller than Svoboda et al.’s by orders of magnitude, and the pooling experiments in Section 3.7 showed, not just by analogy, that adding unlabelled data from nearby stations does not substitute for a larger labelled training set; pooling made results worse in every configuration tested here.

The weak, unfalsifiable version of this conclusion would be that “machine learning needs more information.” The more specific and defensible version is that the constraint on performance here was demonstrably data availability and sensor richness, not an inherent unsuitability of the methods, and this project’s own experiments, particularly the pooling tests, provide evidence for that rather than asserting it by analogy. Whether the gap to CleanSnow’s results could be closed by scaling this methodology up to a comparable number of stations and samples is a reasonable question, but one this project’s resources could not test, and it is left as future work in Section 7.

Blandini et al.’s random forest classifier [4] (Section 2.4) adds an important qualification rather than simply supporting this conclusion. Trained on nearly two million labelled measurements, considerably more than this project had, their model still collapsed to F1 below 0.05 for random-error detection on sensors it had not been trained on, despite F1=0.89 on sensors from its own training network. This matters for how this project’s pooling results should be read: it would be easy to conclude the pooling experiments failed only because six stations and roughly seventeen thousand training

windows is still modest by deep learning standards, and a large enough pool would eventually work. Blandini et al.’s result suggests a less comfortable possibility: that cross-station generalisation for this kind of random-error detection may be a hard problem, unresolved even at far larger scale, rather than something that just needs more data. This project cannot resolve which explanation is correct, but it is the more useful question to leave the reader with.

A second, quieter conclusion is about how machine learning methods get compared, not just which one wins. This project turned up several examples of a single headline metric actively misleading about which method is really better with a modestly sized labelled set: the feature ablation study’s McNemar-versus-F1 inconsistency, the overlapping confidence intervals across all four methods, and the identical aggregate F1 scores a paired test later showed were not equivalent at all. That is arguably as important a finding as the headline result itself, and it generalises well beyond snow sensor data.

7 Future Work

Several extensions to this project were identified but not pursued, either because they represented a substantially larger undertaking than time allowed, or because they were only motivated by findings that emerged late in the project’s timeline.

The most immediately actionable extension is acquiring properly labelled data from additional stations, ideally with the sensor types identified as useful in the CleanSnow work, rather than only pooling unlabelled data as this project did. That would test whether the gap to published, larger-scale results narrows with more comparable data, addressing the open question at the end of Section 6.

A second, more methodologically interesting direction concerns the fixed thresholds used throughout this project. Every method decides whether a point is noise using a fixed multiplier, generally three, of a locally estimated spread, chosen as a standard default rather than tuned against labelled data, so it would not overfit to a small test set. Conformal prediction, a framework originating with Vovk, Gammerman and Shafer and given an accessible introduction by Angelopoulos and Bates (2021) [2], offers a more principled alternative: rather than assuming a threshold in advance, it uses a calibration set to construct a prediction interval with a formally guaranteed coverage property, for example a 99.9 percent interval that is meant to con-

tain a normal reading 99.9 percent of the time, treating anything outside it as anomalous. Applied here, a conformal approach could in principle replace the fixed three-times-spread rule in the baseline, and the fixed contamination or vote-count thresholds elsewhere, with a single, distribution-free rule that adapts its width to observed variability rather than assuming it in advance. One plausible starting point would be using the manually labelled “good” days as a conformal calibration set, exactly the kind of verified-normal reference such methods are designed against. It was not pursued here both because of the time already invested in Sections 3 and 4, and because conformal prediction’s standard guarantees assume calibration and evaluation data are drawn from an exchangeable, broadly similar process, an assumption needing care for a strongly seasonal series like snow depth, and would need addressing properly if this work were taken further.

A third direction, suggested by the results in Section 4.3, would be engineering a feature representing whether a change in snow depth is corroborated by a proportional water-equivalent change, in a more general, continuous form than the single mismatch feature already tested, testing whether this closes the false-positive gap both multivariate methods produced during legitimate storms.

A fourth direction, raised in Section 4.7, would be a properly prospective version built around a temporal holdout: fitting every method only on data up to some cutoff date and evaluating solely against newly labelled days from after it. This was not attempted here since the labelled sample is too small, a generous two-year holdout leaves only 8 of the 64 labelled noise days to evaluate against, so it would require a substantially larger labelled set concentrated in the held-out period, best pursued alongside the additional-station labelling extension above.

8 Self-Assessment

The zero-spread bug in the baseline filter is probably the single moment that shaped how I approached the rest of this project. An early version silently treated the most suspicious possible case, a day disagreeing with an otherwise uniform neighbourhood, as unmeasurable. I only found it because the recall looked implausibly low, so I checked why.

I will admit the scope grew a lot beyond what I originally planned. Two models became three. The single labelled station picked up five pooling stations alongside it, and a single feature set turned into a formally ablated one. Most of that happened reactively, chasing one unsatisfying result after

another, rather than being planned up front. It is not purely a bad thing: the multi-station pooling result and the corrected feature ablation methodology, two of the more defensible conclusions in this dissertation, only exist because I kept following up rather than stopping at the first version. But it did make the timeline hard to predict, and a stricter sense of what was actually in scope would probably have produced a more efficient project, even if a slightly less thorough one.

If I ran this again, I would build the statistical rigour used in the later stages (paired significance testing especially) into the very first evaluation, instead of only reaching for it once the aggregate scores started looking suspiciously close together. Nearly every finding worth reporting here, from each implementation bug to the seasonal confound in the soil moisture features to the corrected statistical test in the feature ablation study, came from checking something I could easily have taken at face value. Doing that from the start, rather than as an afterthought that arrived a bit further into the project each time, would have saved time and made the whole thing more consistently rigorous rather than one that got more careful as it went along.

9 How to Use This Project

9.1 Setting up the environment

All code for this project was written and run locally in Visual Studio Code (VS Code) on my own computer. This section assumes no prior setup and walks through everything needed to run the code from scratch, using either the terminal built into VS Code or a standalone Windows PowerShell window.

1. **Install Python.** If Python is not already installed, download it from <https://www.python.org/downloads/> (version 3.11 or later is recommended) and run the installer. On the first screen of the Windows installer, tick the box labelled “Add python.exe to PATH” before clicking Install; this step is easy to miss but is what lets Python be run from a terminal afterwards. To check whether Python is already installed and accessible, open a terminal (see the next step) and type `python -version`; a message such as `Python 3.12.0` means it is ready to use, while an error means it still needs installing.
2. **Open a terminal in the project folder.** In VS Code: use File → Open Folder... to open this project’s folder, then open the built-in

terminal with Terminal → New Terminal. This terminal opens already located inside the project folder, so no navigation is needed. Alternatively, in Windows PowerShell directly: open PowerShell from the Start menu, then move into the project folder with the `cd` command, for example `cd "C:\Users\YourName\Downloads\Dissertation work"`, replacing that path with wherever this folder was extracted to.

3. **(Optional) Create a virtual environment.** This keeps the packages this project needs separate from anything else installed on the computer, and is not required but avoids version conflicts. In the terminal opened above, run:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

The second command activates the environment; a successful activation adds `(.venv)` to the start of the terminal prompt. If PowerShell refuses to run this second command with a message about “running scripts is disabled on this system”, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` first and then try the activation command again.

4. **Install the required packages.** With the terminal still open in the project folder (and the virtual environment activated, if step 3 was used), run:

```
pip install -r requirements.txt
```

This reads the `requirements.txt` file included in the project folder and installs the exact package versions this project was built and tested with (including pandas, numpy, scikit-learn, statsmodels, TensorFlow/Keras, and matplotlib). This step needs an internet connection and can take a few minutes the first time, particularly for TensorFlow.

5. **Run a script.** Once installation finishes without errors, any script described below can be run by typing `python` followed by its filename, for example:

```
python baseline_filter.py
```

Results print directly in the terminal, and any output files (results tables, plots) are written into the `data` and `figures` folders as described below.

9.2 Running the pipeline

All code produced for this project is written in Python and organised as a sequence of standalone scripts, each corresponding to one stage of the pipeline, intended to be run in the following order from the project’s main folder.

`data_preprocess.py` downloads and cleans raw station data for every station listed in `stations.csv`, a plain spreadsheet controlling which stations are processed without code changes; removing a row and rerunning is enough to change how many stations are pooled later. `label_data.py` is the interactive tool that produced the manually labelled ground-truth set; it does not need rerunning to reproduce this dissertation’s results, since the labels are already saved, but can extend the labelled sample further. `baseline_filter.py`, `arima_model.py`, `isolation_forest_model.py` and `lstm_autoencoder_model.py` each implement and evaluate one of the four methods in Section 3, and can be run independently. `feature_ablation.py` reproduces the systematic feature study in Section 3.8. `denoise_dataset.py` produces the final, imputed dataset in Section 3.10, and should run last. Each script prints results to the console and saves a results table and evaluation plot into `data` and `figures`, so every number quoted here can be reproduced by rerunning the corresponding script.

Three further scripts reproduce figures referenced in Sections 3 and 4 rather than a method’s evaluation. `results_comparison_plot.py` reads the four methods’ already-saved results and draws `results_comparison.png`, needing only the model scripts to have run once first, without retraining anything. `epoch_sweep_plot.py` and `station_pooling_plot.py` do retrain models, at several epoch counts and pooling sizes respectively, taking a few minutes, the second considerably longer because of the larger pooled LSTM runs. Both produce their own results CSV alongside the figure, so their numbers can be checked independently.

A full working example, showing the reasoning behind several individual labelling decisions, is included in `example_labeling_walkthrough.py` for anyone wanting more depth than the written description of the labelling protocol in Section 3.3 provides.

Bibliography

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Ge-

offrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning, 2016.

- [2] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification, 2021.
- [3] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In Jun’ichi Tsujii, James Henderson, and Marius Paşca, editors, *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, Jeju Island, Korea, July 2012. Association for Computational Linguistics.
- [4] G. Blandini, F. Avanzi, S. Gabellani, D. Ponziani, H. Stevenin, S. Ratto, L. Ferraris, and A. Viglione. A random forest approach to quality-checking automatic snow-depth sensor measurements. *The Cryosphere*, 17(12):5317–5333, 2023.
- [5] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.
- [6] François Chollet. Building autoencoders in Keras. The Keras Blog, 2016.
- [7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [8] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, 3rd edition, 2021.
- [9] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

- [10] Christophe Leys, Christophe Ley, Olivier Klein, Philippe Bernard, and Laurent Licata. Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology*, 49(4):764–766, 2013.
- [11] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008.
- [12] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection, 2016.
- [13] Brian O’Neill. Mathematical properties and finite-population correction for the Wilson score interval, 2021.
- [14] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.
- [15] Sebastian Raschka. Model evaluation, model selection, and algorithm selection in machine learning, 2018.
- [16] Mario Roos-Hoefgeest Toribio, Alejandro Garnung Menéndez, Sara Roos-Hoefgeest Toribio, and Ignacio Álvarez García. A novel approach to speed up Hampel filter for outlier detection. *Sensors*, 25(11):3319, 2025.
- [17] Wendy A. Ryan, Nolan J. Doesken, and Steven R. Fassnacht. Evaluation of ultrasonic snow depth sensors for U.S. snow measurements. *Journal of Atmospheric and Oceanic Technology*, 25(5):667–684, 2008.
- [18] Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.
- [19] Mark C. Serreze, Martyn P. Clark, Richard L. Armstrong, David A. McGinnis, and Roger S. Pulwarty. Characteristics of the western united states snowpack from snowpack telemetry (SNOTEL) data. *Water Resources Research*, 35(7):2145–2160, 1999.

- [20] J. Svoboda, M. Ruesch, D. Liechti, C. Jones, M. Volpi, M. Zehnder, and J. Schweizer. Towards deep-learning solutions for classification of automated snow height measurements (CleanSnow v1.0.2). *Geoscientific Model Development*, 18(5):1829–1849, 2025.
- [21] United States Department of Agriculture, Natural Resources Conservation Service. AWDB REST web services documentation and SNOTEL station network.
- [22] University of California, Berkeley, Central Sierra Snow Laboratory. Official website, n.d.
- [23] University of California, Berkeley, Office of the Vice Chancellor for Research. Central sierra snow lab (research unit page), n.d.
- [24] Qingsong Wen, Jingkun Gao, Xiaomin Song, Liang Sun, Huan Xu, and Shenghuo Zhu. RobustSTL: A robust seasonal-trend decomposition algorithm for long time series. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(1):5409–5416, 2019.